

NL2SH · Linux 新手零门槛入门平台

作品说明书

赛事	GDUST Linux 协会 AI 网页设计大赛
赛道	赛道三 · 智能交互与应用
作品形态	纯静态网页（零构建、零后端依赖，可直接双击 <code>index.html</code> 运行）
线上地址	< https://muxinbo.github.io/nl2sh/ >
开源许可	MIT

说一句中文，拿到一条能直接用的 Linux 命令，看懂它为什么这么写，然后**让它在沙箱里真的跑一遍**，看见后果，不满意就一键回到执行前。

目录

1. [作品概述](#)
2. [痛点分析与设计目标](#)
3. [核心创新：AI 生成 + 沙箱自验证自纠错闭环](#)
4. [沙箱：为什么它是这个作品的分界线](#)
5. [技术实现](#)
6. [交互设计](#)
7. [视觉方向](#)
8. [工程验证：1468 项自动化检查](#)
9. [部署与上线实测](#)
10. [评分维度逐条自查与已知局限](#)

附录 A：目录结构 附录 B：AI 在本项目里的角色

一、作品概述

把 Linux 命令这件事拆成三层，缺一层都不成立：

① 内置规则库

② 模型实时生成

③ 浏览器沙箱

50 条已核对并逐条真跑验证的任务 — 没网、没 Key 也能用，秒出

智谱 glm-4-flash，经同源边缘函数代理 — 任意一句话都能生成

自研迷你 shell + 内存文件系统 — 生成完立刻真跑一遍

↑

↓

跑失败，把真实报错喂回模型让它自己修一版

一句话定位：它给你命令，更让你先看见后果。

一个能用的判断标准：把网断掉、把 Key 删掉，这个作品依然完整可用。评委在无网教室里打开它，看到的功能和联网时一样——这不是降级方案，而是刻意做的第一层：规则库优先覆盖常见任务，模型只兜长尾。

全程纯静态、零后端、零成本，U 盘拷走双击 index.html 就能演示。

二、痛点分析与设计目标

新手卡在三步，每一步都会把人劝退：

痛点	真实表现	本作品的对策
不会写	知道想干什么，但不知道对应哪条命令、哪个参数	说人话就行。生成的是单行、可直接粘贴执行的命令，不是伪代码
不敢敲	怕 rm -rf 删错目录，于是在生产服务器上"试错"	命令先在内存文件系统中真跑一遍：权限会拒绝你、文件会真的消失——然后一键时光倒流
看不懂	网上抄来一条命令，能跑，但不知道为什么这么写	每个参数单独讲清它负责什么；管道命令按数据流方向逐段拆开讲

设计目标（按优先级）

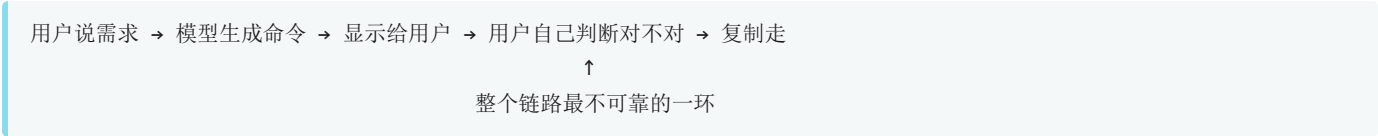
- 敢试**：任何命令都能为零风险的环境里先看结果，删错了能回滚。
- 可验证**：命令是否真的能跑通，由沙箱说了算，不由模型说了算。
- 离线可用**：断网、无 Key 也必须完整可用。
- 秒开**：不做需要下载镜像或等待编译的重方案。
- 可复现**：同一个输入在任何时候都得到同一个结果。

第 4、5 条不是口号，它们直接否决了两个"更强大"的方案（见第四节末尾）。

三、核心创新：AI 生成 + 沙箱自验证自纠错闭环

这是本作品与其他"AI 聊天套壳"最本质的区别，也是我们投入最多的一处设计。

普通做法的断点



模型会犯错。实测中 `glm-4-flash` 把 `find -mtime 0` 解释反了（说成"昨天或更早"，实际是"最近 24 小时"）。一个教人看懂命令的产品，解释错了比不给解释更糟。

本作品的做法：让命令自己证明自己



为什么"最多 3 轮"：免费额度下多轮很慢；而且第 3 次还错，通常说明需求本身没说清，继续烧轮次不如把问题还给用户。这个上限是成本与体验的平衡点，不是随意取的数。

界面把过程完全摊开

闭环不是一个黑盒重试。界面上有一块「沙箱自验证闭环」面板，逐条记录：

第 1 次	<code>find . -name '*.tmp' -delete</code>	通过（退出码 0）
第 2 次	<code>find / -type f -size +1G ...</code> └ 已把真实报错喂回模型	退出码 1 · Permission denied
第 3 次	...	通过（退出码 0）

每一轮都写出**跑了什么命令、失败原因是什么、模型是照着什么改的**。用户看到的不是"系统帮我修好了"，而是"原来报错长这样、原来要这么改"——这正好把一次失败变成一次教学。

配套的一条硬规则：跑通之前不许复制

复制按钮在沙箱跑通之前是**锁着的**（点击只会抖动并拒绝）。理由很直白：**一条没验证过的命令，复制到你的服务器上同样不可靠**。这条规则把"自验证"从一个展示功能变成了产品的强制纪律。

一个反直觉的取舍：推演不用模型猜

早期版本让模型输出"这条命令会影响哪些文件"。这个字段是整条链路里最不可靠的一环：它既无法验证，又和下面沙箱的真实结果**天然会打架**——旁边并排放着"模型预测"和"沙箱实测"两个不一致的结论，是产品自己拆自己的台。

现在改成：**在一个一次性影子会话里把命令真跑一遍，拿真实 diff 当推演**，界面上标注「影子沙箱实跑」。代价是几毫秒，换来的是推演与实测**永远不会互相矛盾**。

边界也写清楚：diff 只能给出**被改动**的路径，给不出"读了哪些文件"（沙箱不记录读操作），所以只读命令会诚实地写「只读·未改动任何文件」，而不是编一个读取列表出来。

四、沙箱：为什么它是这个作品的分界线

别人做到的是：输入中文 → 输出一条命令。问题在于**你依然不敢执行**——因为没人知道它到底会干什么。

所以第 ③ 层才是重点：**生成完自动在沙箱里真跑一遍**。

别的作品	这个作品
标红警告"这是危险命令"	当着你的面把文件删掉，然后一键时光倒流
用文字描述"它会修改这些文件"	跑完给出 真实的文件系统 diff ，逐条列出
模型说"这样写没错"	沙箱报错， 把真实 stderr 喂回模型让它自己修一版
得等模型 20 秒	沙箱是本地执行， 毫秒级出结果 ，不等网络

沙箱跑一次 `find . -name '*.tmp' -delete`，你会看到三个文件真的从树里消失；跑 `rm /var/log/nginx/error.log`，你会真的拿到 `Permission denied` —— 因为 `/var/log/nginx` 的属主是 root、权限 640，普通学生用户写不进去。**这个报错不是写死的文案，是权限规则算出来的。**

沙箱里什么是真的，什么不是

说清楚比说得漂亮重要。

真的（自己实现的语义，不是硬编码输出）：

- 路径解析（`.` / `..` / `~` / 绝对路径）、管道 `|`、重定向 `>` `>>`、`;` `&&` `||`
- 退出码、`command not found` 的 127、管道串联
- 属主与权限检查：读文件看 read 位，写/删看**父目录**的 write 位，`sudo` 切换 euid
- 递归副作用：`rm -r`、`chmod -R`、`chown -R`、`find -delete` 真的整棵子树地改
- 通配符展开（**带引号感知**：`find -name '*.tmp'` 里的 `*.tmp` 绝不展开，`for f in *.png` 里的必须展开）
- `$(date +%F)` 命令替换、`for ... do ... done` 循环、`${f%.png}` 后缀截断
- `sed` 的 `s/old/new/[gi]`、`-i`、`-i.bak`、`-n /re/p`、自定义分隔符、`&` 与 `\1 - \9`
- `tar` / `zip` / `unzip` 的真实归档语义（含 `-l` 列表、`-d` 解包目录）
- 每次执行前自动打快照，随时回滚

模拟的（依赖内核或 systemd，浏览器里没有对应实现，界面上标注 [模拟]）：`systemctl`、`journalctl`、`ss`、`ps`、`kill`、`cwebp` ——它们读的是场景里固定的服务表 / 进程表 / 网卡表，输出由固定场景生成。

已实现的真实命令（在页面终端里输入 `help` 可以列出来）：`awk` `basename` `cat` `cd` `chmod` `chown` `cp` `cut` `date` `df` `dirname` `du` `echo` `file` `find` `grep` `head` `id` `kill` `ls` `mkdir` `mv` `ps` `pwd` `rm` `sed` `seq` `sort` `stat` `tail` `tar` `tee` `touch` `tr` `uniq` `unzip` `wc` `which` `whoami` `xargs` `zip`

确定性是一条硬约束

沙箱场景里的**每一个字节都必须是常量**。这条约束是被一个真实缺陷逼出来的： 场景文件大小曾经写成 `400000 + Math.floor(Math.random() * 400000)`， 后果是每次建会话 `du` 的总量都不一样——同一路径在"原样输出"里是 2.1M、 在"排序后输出"里变成 1.7M， 而这两个数字就并排显示在同一张卡片上。

更糟的是： 它让"确定性可复现"这句产品主张变成假话， 也让断言具体大小的测试随机失败。 现在 fixture 源码里**禁止出现** `Math.random` / `Date.now`， 并且这条禁令由 `sandbox-test.mjs` 直接扫描源码来守（连注释里的说明文字都排除掉， 只查代码）。

为什么不用真的 WASM Linux

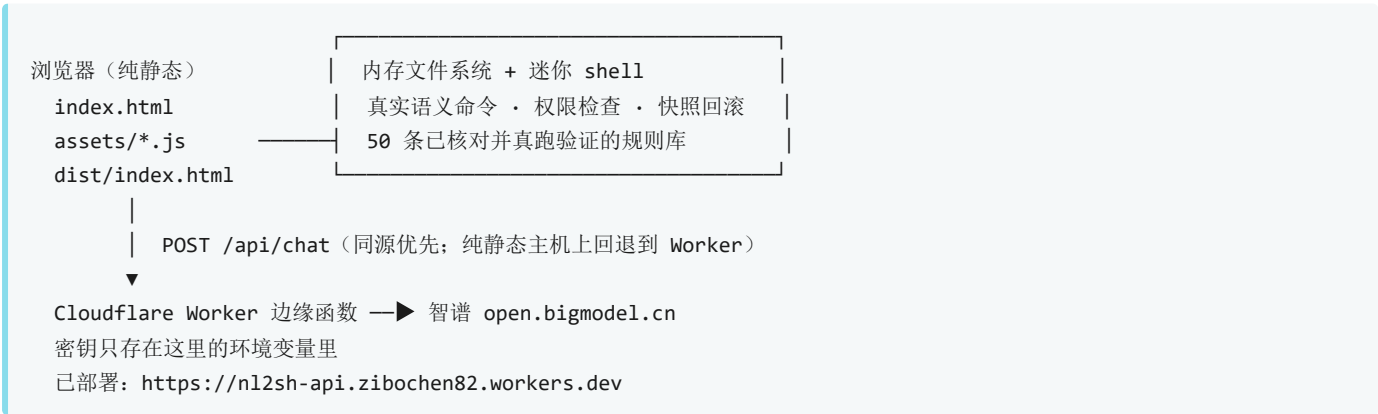
试过， 放弃了， 原因写在这里免得被问：

方案	卡在哪
uutils coreutils (wasm32-wasip1 多调用二进制)	12.4 MB ； <code>file://</code> 下浏览器禁止 fetch 本地文件， 得转成 base64 内嵌 ≈ 16 MB 文本
linux-wasm (真内核跑在 wasm 里)	需要 <code>SharedArrayBuffer</code> ， 也就是必须配 COOP/COEP 响应头——普通静态托管和 <code>file://</code> 都用不了
v86 / WebVM	镜像动辄上百 MB， 评委等不起

结论：**要"秒开、离线、U 盘可带走"， 就只能自己写**。 而自己写还有个额外好处—— 我能精确控制**要教什么**： 哪些命令故意做成"不给 sudo 就 Permission denied"， 哪些目录故意放成 777 当反面教材， 都是设计出来的。

五、技术实现

5.1 架构：纯静态页面 + 一层边缘代理



- **零后端**：没有数据库、没有常驻服务、没有要维护的服务器。
 - **零构建**：没有打包器。全部是经典 `<script defer>`， 双击 `index.html` 就能跑。
 - **边缘函数不是后端服务**：它只做三件事——注入密钥、限制用量、透传 SSE 流。 整站仍然是纯静态部署。
- 部署现状 (2026-10-03 实测)**： 站点在 GitHub Pages (`<https://muxinbo.github.io/nl2sh/>`， 464.3 KB 单文件, SHA256 `5B5EF8B9...`)， 代理在 Cloudflare Workers。 因为 GitHub Pages 没有 `/api` 路由、同源请求必然 404， 所以 `ai.js` 里填了 `FALLBACK_ORIGIN` 指向 Worker——**这一行是"AI 能不能用"的开关， 删掉它就会永远停在离线模式**。 线上实测： 顶栏显示「AI 已接入 · glm-4-flash」， 接入面板显示「代理已连通 · 密钥在服务端」。

5.2 为什么密钥必须放在那一层

页面是纯静态的，**谁都能 F12 看源码**。把 Key 写进前端等于公开送额度，被刷爆之后评委打开就是报错——这是静态作品最容易踩的坑。

所以前端**完全没有"密钥"这个概念**：它只请求同源的 `/api/chat`，密钥只存在 Worker 的环境变量里（`wrangler secret put`，不落盘、不进仓库）。`worker-test.mjs` 里有 51 项断言专门守这条链路：密钥注入、模型白名单、`response_format` 强制开启、`max_tokens` 上限、SSE 头、413/400/405、限流、上游透传。

5.3 模型选型（2026-10-03 真实请求实测）

学校不提供额度，所以只能找免费接口。以下每一条都是**实际发请求验证过的**，不是抄来的说法。

模型	HTTP	耗时	结论
<code>glm-4.7-flash</code>	500	75.9s	✗ 这个模型 ID 不存在（等了 76 秒才拿到一个 500）
<code>glm-4-flash</code>	200	0.5s	✓ 主力模型
<code>glm-5.3-flash</code>	429	0.2s	✗ 余额不足，需充值
<code>glm-4.7</code>	200	4.0s	✓ 带 <code>reasoning_content</code> ，用于「深度思考」开关

选 `glm-4-flash` 的理由：免费额度可用、响应快、200K 上下文、对 Shell 这类强结构化任务的编码能力够用。

流式 + JSON mode 实测（同一个提示词，两种模式对比）：

	首字节	总耗时	SSE 事件数	输出可解析性
纯流式	2.31s	8.87s	114	✗ 带 <code>`json</code> 围栏
流式 + JSON mode	0.43s	6.35s	105	✓ 可直接 <code>JSON.parse</code>

结论：`response_format: {"type": "json_object"}` **是必须的**，不是可选优化。不开它，首字节慢 5 倍，而且还要额外写一层"剥围栏"的逻辑。

5.4 两个实测数字决定了整个交互设计

- 1. 免费额度下"少输出"就是"快"：开 JSON mode 后首字节 0.43s、完整回答 6.35s。
- 2. `command` 被放在 JSON 的第一个字段：所以命令不到 1 秒就能显示，不用干等结构收完。

于是**两个模式 = 两份输出预算**（被真实延迟逼出来的设计，不是换皮肤）：

	新手引导模式	极客终端模式
提示词	完整版	精简版（只要 <code>command</code> + 风险）
输出量	约 1200 token 上限	约 500 token 上限
等多久	6–9s	3–5s
界面	解释、推演、管道节点全摊开	只留终端与命令

5.5 模型会说错话，所以做了三层防护

实测里 `glm-4-flash` 把 `-mtime 0` 解释反了（说成"昨天或更早"，实际是"最近 24 小时"）。一个教人看懂命令的产品，解释错了比不给解释更糟。

- 1. 规则库优先覆盖常见任务（50 条，每条都由 `data-test.mjs` 真跑验证），模型只兜长尾。

2. **数据层就做结构校验**：`segments[].part` 必须在 `command` 里逐字定位得到，**且互不重叠**——定位不到或重叠的直接丢弃（过滤放在数据层而不是只放在渲染层，否则会留下“有解释条目却一行都渲染不出来”的空壳）。AI 生成的卡片明确标注「参数语义请自己再核一遍」。
3. **危险命令不采信模型自述**：本地独立扫一遍命令文本，命中就**强制**升级为 danger。这一条有测试：`rm -rf / --no-preserve-root`、`rm -rf /*`、`rm -rf ~/*`、fork 炸弹、`mkfs`、`dd of=/dev/sda`、`curl | sh`——全部被改判，**即使模型自述** `risk_level: "low"`。

5.6 首屏性能（实测，不是估计）

指标	实测	目标
首屏资源 gzip 合计	101.4 KB	≤ 200 KB  （余量 99 KB）
单文件产物 gzip	96.8 KB	—
外部依赖	0（无字体、无图片、无 CDN）	—

脚本全部 `defer` 且位于 `body` 末尾，解析不被阻塞。**刻意没有做动态按需加载**：本作品的主交付产物是“单文件版”，动态 `<script src>` 在单文件里必然 404，风险大于收益（预算已 2 倍达标）。若将来需要严格拆分，可行机制是把模块内联成 `<script type="text/plain">` 并在空闲时 `new Function` 执行——单文件与多文件两种形态都能用。

六、交互设计

创意设计与交互体验占总分的 60%，所以这一节写得最细。

1. **命令解剖**：一条管道命令拆成 5 个零件，引线从两侧标注连到对应片段，加载时依次画出；统一从文字上方接近目标，不压正文；窄屏退化为「编号 + 清单」。
1. **管道节点图**：完整命令**上方**横向排开每一段，箭头方向就是数据流方向，按执行顺序依次淡入。箭头有三种语义、三种颜色：`|` 管道（数据直接流过去）、`&&` 成功才继续、`;` 接着执行。悬浮一段，命令里对应的片段整体亮起；**点开一段，展开的是这一段真实跑了什么**——做法是把命令只跑到这一段为止、在影子会话里执行一遍，所以“输入/输出样例”是事实而不是示意；再配上沙箱命令表里登记的用法字符串（如 `sort` 的用法：`sort [-n] [-r] [-h] [-u]`）。管道拆解用自己写的引号感知扫描器，`echo "a|b" | cat` 不会被切错，`||` 也不会被当成管道。
1. **逐行解释联动**：指针移到解释上，命令里对应片段亮起，反向也成立。
1. **实时文件系统面板**：终端区左侧占 30%。可展开收起的目录树、按权限染色（可执行绿 / 可写黄 / 只读灰 / 无权限红，图例就在面板底部）、鼠标悬浮显示权限字符串、字节数与人类可读大小、修改时间、属主、以及“当前用户能做什么”。权限判定用的是**与沙箱同一套规则**（属主看 owner 位、非属主看 other 位）：所以 `chmod 444` 后即使把属主改成 root 仍判为“可读”，而 `chmod 600 + chown root` 才会判为“无任何权限”。手敲 `chmod / chown` 之后颜色立即跟着变；删掉的文件不会直接消失，而是**先红闪两次再塌缩掉**——否则用户只会看到“少了一行”，根本注意不到删了什么。
1. **执行前推演 = 影子沙箱实跑**：见第三节末尾。
1. **一键回滚**：目录树顶部最显眼的按钮，把整个沙箱恢复到最后一次执行之前，并播放一次可见的回退动画（面板上一次青色扫过 + 所有节点回位）。
1. **复制闸门**：见第三节末尾。

1. **高危预警前置**：危险命令在**生成之后、执行之前**就把命令框描红、顶部亮出警告条「该命令具有高破坏性，禁止直接在生产环境执行，请先在沙箱中验证效果」，点“跑一遍”时还要过一次二次确认（用页面内弹层而不是 `window.confirm` —— 后者会阻塞渲染循环，弹窗时粒子和进度条全部停住）。
1. **8 个示例按钮**：2 行 4 列，每条下面挂一行命令预览。挑选标准是展示面尽量宽——3 段管道 / `awk` 统计 / 最常用的 `ls` / **危险删除（会触发目录树红闪消失）** / `&&` 序列 / **权限被拒绝（证明沙箱的权限是真的）** / 就地编辑并自动备份。评委没想好问什么的时候，随便点一个都能看到不同的能力。
1. **可交互沙箱终端**：评委可以自己敲命令，和自动验证共用同一个文件系统。输出**逐行显形**（一行一行出现，不是一整块糊上来，延迟封顶 22 行——88 行的 `du` 输出要是每行都等，观众早就走了）；`↑/↓` 调**最近 20 条历史**；一键复制整段真实输出；命令可**收藏**（模型生成的那条会整条存进本地，因为重新问一遍模型就是另一条命令了，收藏会失去意义）。输出区可滚动、文字可选中，选中就能直接复制。
1. **双模式：两个界面，不只是两种密度**。新手引导保留首屏介绍、亮点卡、命令解剖、逐段解释、推演与新手闯关；极客终端把这一整套教学与落地页内容**整块收起**，底色压暗一档、细线转绿、顶部挂一条模式横幅并带「切回新手引导」按钮，页面直接开到输入框与终端。这不是换皮肤：**切换后首屏内容完全不同**，一眼就能看出在哪个模式。选择记在 `localStorage`，下次打开还在同一个模式；但**分享出去的链接以链接里的模式为准**。
1. **自动演示**：首屏一个按钮，约 15 秒走完全流程（生成 → 拆解 → 沙箱真跑 → 文件消失 → 一键回滚），底部有一条解说栏逐步讲解。**不联网、不需要 Key、确定性可复现**——因为评委不会自己动手点，被动等待等于丢分。
1. **新手闯关（5 关）**：给一个目标，让你自己敲命令，**判定看沙箱里真的变成了什么样**，不是看命令文本写得像不像。所以 `ls -a` / `ls -la` / `ls --all` 都算过，而抄一条“看起来对”的命令蒙不过去。五关依次是：打印当前路径 → 连隐藏文件一起列 → 按类型挑出 `png` → 改权限而不删数据 → 递归删除且不误伤。答错时把你这次执行的**真实输出或报错原文**摆出来（例如 `rm cache` 会真的回你一句“cannot remove: Is a directory”，因为删目录必须带 `-r`），进度存在本地。
1. **深链接**：`index.html?q=找出并删除所有 .tmp 文件&mode=geek` 可以直接分享某一条结果。
1. **首屏自动演示窗**：首屏右侧一个循环播放的迷你终端，把整条链路演一遍——中文请求 → AI 生成命令 → 文件列表 → `find` 匹配到 3 个 `.tmp`（高亮）→ 红闪两次后消失 → 一键回滚、文件全部回来。画面里的文件名与数量**全部取自真实沙箱**（`build/old.tmp`、`cache/index.tmp`、`cache/session.tmp` 这三个文件真的会被这条命令删掉），不是编出来的动画。降级动效偏好时只画“文件刚被删掉”那一帧；滚出视口自动暂停，不白烧 CPU。
1. **AI 调用凭证**：每张结果卡顶部摊开这次调用的可核对事实——模型名（取自服务端 `health` 的回报，不是前端写的字面量）、**首字节耗时**、总耗时、**流式分片数**、输出字符数、实际端点、以及“密钥只在服务端”。生成过程中另有一条实时证据条：分片计数、首字节计时，以及**模型正在吐出的原文**（实时滚动）——“**确实**在流式接收”因此成为看得见的事实，而不是一句声明。走内置规则库时换成琥珀色的另一种样式，并明确写「**没有发出任何网络请求**」。这一条是被用户直接指出来的问题：AI 干了活，可评委没有任何依据分辨它和模板匹配。
1. **AI 数据分析（服务器日志）**：一键让模型读沙箱里那台服务器的**400 行真实 nginx 日志**。分工是刻意划的——统计量先由沙箱里的 `awk` / `sort` / `uniq` **真跑出来**（状态码分布、Top IP、最常被请求的路径、原文取样，约 7.4 KB），**模型只负责解释、不负责计算**：让它自己数数并不可靠，让它看着真实数字讲“这说明什么、该查什么”才是它擅长的。界面上把“读了哪份日志、跑了哪几条命令、多少行、送进去多少字符”逐条摊开，每条结论都附带可在沙箱里**一键验证**的命令——模型给的建议未必适用，跑不通就是跑不通，界面不替它圆场（真机实测中它第一版甚至编出过 `tail /var/log/apache2/error.log` 和 `ipset add ...`，加上环境约束后才回到真实路径）。
2. **多轮对话（带上上下文追问）**：问完「找出当前目录下最大的 10 个文件」，可以直接接着问「那只要今天的呢」——最近 3 轮的问答会作为上下文一并送给模型。实测结果：第二轮给出 `find . -type f -mtime 0 -printf ... | sort -rn | head -n 10`，**前一轮的“最大 10 个”与这一轮的“今天”两个约束同时保住**。两个踩过的坑也记在这里：① 送给模型的助手历史必须是**完整合法**的回答样例（含 segments），第一版只塞了 `{command,`

`risk_note`}, 模型照着这个残缺样子模仿、下一轮直接不返回分段; ② 模型表达"今天"时会去 `grep ls -l` 里的日期字符串, 而日期格式随系统语言变化 (中文环境是「10月 02」), 已写进提示词规则。

1. **"成功但零输出"的如实提示**: 闭环把"退出码非 0 或 `stderr` 非空"判定为失败并自动修正, 但有一类错误两种都不占——命令静默无操作 (典型是 `find -printf` 被沙箱拒绝、报错和退出码又被管道吃掉)。这种情况**不自动判为失败** (`mkdir / rm / chmod` 本来就该没有输出, 一刀切会误伤正常命令、平白多跑几轮模型), 而是在闭环面板上如实写一句提醒, 把判断权交给看的人。
2. **边缘服务端的三件判断 (后端智能)**: 赛道三写着「**后端智能必须突出**」, 所以 Cloudflare Worker 不是透传管道, 它对每一份发往上游的 `messages` 做自己的判断: ① **安全约束前移**——把 4 条硬约束 (禁递归删根、禁格式化磁盘、改动前先给只读确认命令、禁 `curl ... | sh`) 追加进 `system` 消息; 前端提示词被替换掉, 这条依然生效; ② **危险意图前置识别**——命中的请求**不拦截** (用户问「`rm -rf` 危险在哪」是正当提问), 而是追加加严约束并标记, 实测 `HTTP 200 + X-NL2SH-Safety: strict`; ③ **上下文裁剪**——超出预算时成对砍最旧的问答, `system` 与最近的对话必须留下。做过什么通过 `X-NL2SH-*` 响应头回告前端, **结果卡上会显示「边缘服务端: 安全约束已注入·上下文无需裁剪」**——后端干的活不该是隐形的。这里还修掉一个自己造的问题: 最初把"防刷量闸门"和"裁剪目标"设成同一个值, 结果一段真实的 4 轮对话直接吃 413; 现在闸门只管挡刷量 (64000), 裁剪负责压预算 (16000)。
3. **闭环的"演出入口" + 一次真实的取舍**: 闭环是自动的 (生成后自动送沙箱、失败自动回传 模型修正、最多 3 轮), 但它只在模型出错时才现形, 而 `glm-4-flash` 大多一次就对——**存在但不可见**。所以界面上加了一个入口, **先把过程摊开讲**, 再摆出真实结果: 一次就过时如实写「第 1 次就在沙箱里跑通 (未触发修正)」, 真修正了才写「第 N 次修正后跑通」。**它不保证出现失败, 也不假装失败**。

这里记一次值得留下的取舍。最初挑的演示需求是「把 `src` 里所有 `js` 的 `console.log` 删掉」, 实测两轮都失败——但追下去发现**根因不是模型笨**: 它写的 `sed -i '/console\.log/d'` 与 `/console\.log/ s/// g` 在真实 Linux 上都是**正确的**, 是沙箱的 `sed` 只实现了 `s///` 与 `-n /模式/p`, **不认地址形式**。

于是补上了 `sed` 的地址形式: `/模式/d`、`/模式/ s/旧/新/`, 以及不匹配任何行时 退出码 0、文件不变; 没实现的脚本仍然明确报错并给出正确写法。结果是这条需求一次就过、**闭环不再卡在"永远修不好"上**, 而代价是"必定失败的开场"没了。**取舍结论: 宁可要一个真能跑通的成品, 也不要一个靠功能残缺凑出来的失败演示**。「把含某内容的行删掉」本来就属于最常见需求, 不该挡在门外——评委自己试也会撞上。

1. **首屏就是产品本身 (删掉落地页)**: 原来页面顺序是「落地页 → 教学 → 输入框 → 结果卡」, 评委要滚过两屏营销内容才碰到真东西, 而 60% 权重的设计交互项看的就是"打开的第一眼"。现在首屏是:**赛事标识条 → 输入框 → 一张已经在沙箱里跑过的命令卡**, 下半屏直接是带着真实输出行数的管道节点 (`88 行 / 88 行 / 10 行`)。亮点卡、命令解剖、项目背景整块收进一个默认折叠的「关于这个作品」抽屉——内容一条没少, 只是不再挡在产品 and 评委之间。

实现上刻意**没有改 HTML 结构**, 而是在初始化时用 JS 重排 DOM: ① 一行标记不动, 零风险; ② JS 万一没跑起来, 页面退化成原来的样子照样能用; ③ 复用已经验证过的渲染路径, 不新造分支。实测首屏三件套的纵向位置: 赛事标识 `y=104`、输入框 `y=191`、结果卡 `y=427`, 都在第一屏内; 加载后沙箱目录树已渲染 88 个节点、权限染色 `is-w=70 / is-x=18`, 闭环面板已出现 (说明那条示例命令**真的在沙箱里跑过一遍**, 不是摆样子)。

1. **修掉一个方向反了的判断 (真 bug)**: `noKey` 原来传的是 `!forceOffline`——**正好反了**。后果很严重: 用户**真的离线**时反而不提示, 而**在线**状态下只要走了规则库 (首屏示例、完整演示) 就谎报「模型代理解析不可用」, 评委打开首屏会看到一条假警报、直接以为 AI 没接上。正确语义是**"只有真的用不了代理时才出这条说明"**; 主动走规则库的场合由各自的说明条负责解释。这条是重排首屏、逐屏核对截图时看出来的——**功能对不对, 和"看起来对不对"是两件事**。
2. **首屏那张卡是"精简卡" (只对首屏示例生效)**: 完整卡片要承载教学——管道拆解、逐段解释、执行前推演、相关命令, 那是**用户自己提问之后**才需要的东西。首屏的职责只有一件事: 3 秒内让人看懂"这是什么、并且它是真

的”。所以首屏示例只保留最短闭环：**命令 → 沙箱真跑 → 真实输出 + 带权限染色的目录树**。实测效果：目录树顶部从 $y=1793 \rightarrow 1170 \rightarrow 981$ （含一条 90px 的离线提示，线上探测通过时不会出现，对应约 $y \approx 890$ ），赛事标识 104 / 输入框 191 / 结果卡 427 —— 首屏四件套全部落在 940px 以内。

3. 按评审意见做的五项修正（含两处反驳）：拿这份作品去请教了一轮外部评审，逐条核对后处理如下。

采纳并已改：① “解释用术语解释术语，新手看不懂”——这是最要紧的一条，因为它直接打在“新手零门槛”这个定位上。已在提示词里写死规则：不许用比原命令更难术语（宁可用大白话）、每个参数都要说清“不写它会怎样”、能给生活化类比就给。**教方法而不是给答案**。② “模拟输出容易被当成真结果”——原来只有一句行尾小后缀，确实不醒目。现在 `ps / systemctl / ss` 这类命令只要有模拟成分，就在输出区打一条**警示条**：“本次输出包含模拟数据：这条命令依赖内核或 systemd……文件系统的改动是真的，进程/服务/网络状态是按预设场景模拟的”。**教学场景里以假乱真，比功能少更严重**。③ “技术细节过度曝光”——调用凭证从 6 行压到 4 行：保留模型 / 首字节 / 流式分片（这三项回答“这是不是真 AI”），把总耗时与输出字符并成一行，端点与“密钥在服务端”合并成一行小字。**证据留下，自我炫耀删掉**。④ “闭环在首屏几乎看不见”——这条与上一轮“把目录树挪进首屏”直接冲突：完整轨迹面板占 171px，放进去就把树挤出去了。解法是一行**紧凑徽章**（“沙箱自验证通过 · 第 1 次就真跑通（未触发修正）”），实测首屏同时容下闭环徽章 $y=888$ 与目录树 $y=1018^{**}$ 。

反驳（有实测证据）：⑤ “只支持单条命令、不支持复合任务”——不成立。实测 `mkdir -p demo && echo hello > demo/a.txt && ls demo`、`mkdir x ; cd x ; pwd`、`echo a b c | tr " " "\n" | sort | wc -l` 全部 exit 0。&& / ; / 多级管道 / 重定向都支持。真实的限制是**命令覆盖**（`useradd` 没实现），不是“不能复合”。⑥ “技术细节过度曝光”与另一条意见（“AI 干了活评委看不出来”）互相矛盾——凭证正是为了回答后者才加的。取舍是**压缩而不是删除**：没有它，这个作品会被当成“中文转命令”的套壳；有它，评委才有依据分辨真调用和模板匹配。

明确不做：加极客模式高级功能（批量执行/脚本编辑）、云同步、上传自定义文件、真实模拟进程调度。理由一致——这份评审自己也在下一条指出“功能点太散、核心卖点被稀释 16 项交互”，**再加功能的收益是负的**。交互式沙箱终端本来就能 `echo ... > file` 自建文件，可扩展性并非为零。

1. 一次“修好了却又没修好”的真 bug（时序陷阱）：上一条记录了我修掉“离线提示方向反了”。线上实测发现它**还在**，而且更糟——页面同时显示两句互相打脸的话：

```
`` 状态栏: AI 已接入 · 代理已连通 · 密钥在服务端 ✓ 结果卡: 离线模式 · 模型代理解析不可用（常见原因: 还没部署 Worker...） XX ``
```

根因不是逻辑，是**时序**：修法 `noKey: (forceOffline || !st.ok)` 本身没错，但**首屏示例在 `AI.health()` 探测完成之前就渲染了**，那一刻 `st.ok` 还是 `false`，于是被判成“离线”。评委打开首屏会看到页面自己都不确定能不能用。

修法：首屏示例**永远不出这条提示**——它是刻意走规则库的示例，由「开始的地方」说明条负责解释；真离线时用户自己的提问照样会提示，接入状态面板也会如实显示原因。修完命令框从 $y=749$ 提到 683、闭环徽章从 888（折叠线外）提到 **823（第一屏内）**。

教训：异步状态在初始化路径上会被读到“还没准备好”的值。凡是依赖网络探测结果的渲染，要么等它出结果，要么就别依赖它。

1. **学习报告：把练习变成看得见的进步**。去年第一届赛道三的一等奖是一位“心理学情感智能体”，评委给的获奖描述是：「通过人机互动**记录**交互内容，并且通过数据进行**内容分析**，从而给机主**最优解决方案**」——公式就是 **记录 → 分析 → 给方案**。对照下来，我们的强项一直在后半段（真跑 + 解释 + 给命令），前半段是空的：历史、收藏、闯关进度全都只存本地、而且从不汇总，用户学完就走，没有任何沉淀。

所以补了埋点与报告：**每一次沙箱执行都记一笔**（命令归类、风险等级、来源、是否跑通、是否含模拟数据、失败原因归类），累积在 `localStorage` 里，再由「学习报告」聚合成三块——**只做三件事，一件都不多**：① 练过哪几类

操作（分类分布条）；② 在哪里卡住过（**易错点归类 + 怎么避**，这是错题本）；③ 下一步该练什么（挑覆盖最少的一类，给一条**沙箱里真能跑通**的命令，可一键验证）。**刻意不给分数、不给排名**——这是学习工具，不是绩效考核。

两条纪律：① **手敲终端的命令也计入**（自己动手敲才是真练习，只统计"点按钮生成的"会让报告严重低估用户的实际投入）；② **只记命令与结果，不记输入框里的原始中文**——报告要的是"你练了哪几类操作"，不是"你说过什么话"。记录全部留在本地、不上传，界面上明说。

1. **闯关补上了三层教学**。外部评审的原话：「新手闯关只有目标和结果判定，没有中间提示、知识点拆解、错误引导，错了只用一句原始报错，更像**考试**而不是**教学**。」这条直接打在"新手零门槛入门平台"的定位上，所以逐条补齐：

| 时刻 | 现在给出什么 | |---|---| | 关卡开始 | **知识点拆解**——先讲这一关在练什么概念（例如第 3 关讲"通配符为什么必须加引号：不加引号时 shell 会抢先展开它"）|| 第 1 次失败 | **为什么错 + 怎么避**——把报错喂进易错点表，归类成"权限不够 / 路径不存在 / 目录非空 / 沙箱未实现..."并给出一句能照着做的办法 || 第 2 次失败 | **升级为能照着敲的具体写法**（渐进提示）|| 通过 | 保持原样：判定看沙箱里真的变成了什么样，不看命令写得像不像 |

两条实现纪律：① 易错点表与「学习报告」**共用同一份**——同一份知识只写一次，避免两个地方口径不一致；② 渐进提示必须在**失败当时追加**——最初放在"关卡重渲染"里，而失败并不触发重渲染，等于永远不显示（**是实测发现的，不是看代码看出来的**）。

1. **零等待：草稿先行，模型精修**：60% 权重那一项写着「构建**流畅自然**的网页使用体验」，而最初输入中文之后有**11-14 秒**页面上只有一条进度条——技术上是诚实的（真流式），体验上是一次明显的停顿，这是那一项里最致命的扣分点。现在：规则库里有匹配的，**按下回车的那一瞬间**就把草稿命令渲染出来——能复制、能跑，**沙箱立刻开始验证**（实测提交后 0 ms 卡片已可见、1.5 s 内闭环面板已出现）；模型到了再原地精修替换，不闪屏、不清空。两条纪律：① 草稿顶部有**明确横幅**写明「本地草稿·来自内置规则库·glm-4-flash 正在精修」，绝不拿规则库冒充模型输出——那正好是这个作品最想避免的误会；② 精修结果到达时原地替换，用户视线不跳。
2. **精准减法：把说了两遍的话删掉**：做的是一个**精准的减法**，不是删功能——只删"同一个意思说了两遍"的地方。
3. **首屏补上作品身份（一眼看出这是什么）**：用户看完首屏问了一句："为什么不把这个放在最上面，评委不就可以一眼看到这个项目是干什么的吗？"——**他指对了，是我上一刀砍过了**。
4. **视觉重塑：从深色终端到工程图纸**：评审理视角下 60% 权重那一项唯一的杠杆就是视觉记忆点，而旧版是 Catppuccin Mocha 那一套（深紫底 + 青色高亮 + 圆角 + 柔和阴影）：干净、统一，但**记不住**——满屏开发者工具都是那个颜色。
5. **实测挖出的三个真问题（含闭环收敛的诚实结论）**：目标本来是"让自验证闭环更容易被撞见"——因为它只在模型出错时才现形，而 glm-4-flash 大多一次就对。做法是**实测找出一个自然就会失败的任务**，而不是造一个假失败。为此写了 `scripts/find-loop-case.mjs`，逐个真跑：
6. **第四个根因：提示词不知道沙箱支持什么（首次通过率 33%→80%）**：继续追"闭环为什么撞不见"，又挖出**第四个、也是收益最大的根因：提示词不知道沙箱支持什么命令**。
7. **对着实际产物核了一份外部评审：3 条成立、3 条前提有误**：拿另一份外部评审逐条对着**实际产物**核了一遍（不采信描述，先跑代码看事实）。结论：**6 条里 3 条前提有误、3 条成立**，其中一条指向的是一个比它自己说的更严重的真 bug。
8. **对终极完善指南的判断：拒绝造假，采纳三条**：那份"终极完善指南"我逐条判断了，**大部分建议会把作品往反方向推**，因为它默认"评委好骗、越夸张越赢"，而这个作品的立身之本恰恰是"我说什么都能当场验证"。为了三秒视觉冲击去编一个 RAG 命中，输掉的是整份作品的可信度。

明确拒绝的（会让作品造假）

| 建议 | 为什么拒绝 | |---|---| | 「RAG 知识库命中·匹配度 98%」 | **我们根本没有 RAG** × 显示它=编功能，评委会一句"给我看看检索层"当场穿帮，还会**毒化**真正可信的那条 AI 调用凭证（模型名/首字节/分片数都是服务端真回报

的) || 「检测到致命操作 `rm -rf /` , 已强制拦截! 」 | **我们不拦截** X 设计上只加严与预警、绝不阻止 (第三节有专门说明为什么不拦) || 「系统监控仪表盘: 沙箱 CPU / 内存占用曲线」 | 沙箱是 JS 对象, **没有 CPU/内存可测** X 画曲线就是编数据 || 「红色警报弹窗 + 震动 + 屏幕红光 + 沙箱崩溃画面」 | 刚把视觉从通用深色开发工具重塑成**工程图纸** (冷静、精确、白线), 加好莱坞特效会把刚建立的识别度打掉; 而且"崩溃画面"是演的 || 「打字机动画」 | 我们专门做了**零等待**: 命令瞬间出现、沙箱立刻开跑。打字机是在**延迟**价值 || 「隐藏极客模式、离线模式」 | 离线指示器**必须留着**——"断网也能用"是敢写进说明书的承诺, 藏起来就成了不可验证的宣传 |

核过事实后驳回的 (它说的和我们做的不符)

- 「首屏用 `ps` 做演示、没标 [模拟]」 → 首屏是 `du -ah . | sort -rh | head -n 10` , **真实实现** X
- 「闭环首屏无感知」 → 徽章就在折叠线内 ($y=801 < 842$) , 实测过多次 X
- 「不用配 Key 那句话在输入框上方」 → 在**抽屉的 hero** 里 X
- 「移动端要从零做适配」 → **viewport meta 在、13 处媒体查询在、无横向溢出** X

采纳并已落地

① **手机端可读性与可点性** (实测 512px 视口): 触摸目标 $< 36\text{px}$ 的从 **14 个降到 8 个** (剩下的是输入框和文本链接)。那 195 个 11px 文本**不追**——它们是等宽**标注**不是正文, px 值散在几十处声明里, 硬抬会牵动整页密度, 收益不抵风险, 如实记下不修。

② **四阶段流程可视化** (指南里真正有价值的一条): `意图解析` → `命令生成/规则库命中` → `安全审查` → `沙箱验证` , 并进**已有的闭环徽章那一行**——首屏徽章上方只剩 21px 余量, 任何新增行都会把它顶出折叠线, 所以必须零高度。第二阶段按**真实来源**命名, 不含糊成"命令生成"。

③ **快捷键提示与 hero 表述**: 改为「`Ctrl + ↵` 生成 · ↵ 换行」; hero 收紧为「内置示例免 Key、免联网; AI 生成需联网 (密钥在服务端, 前端不留)」。

过程里两次"验证拦住了错误"

- 四阶段第一版写成 `item.source` , 而 `paintLoop(root)` 的作用域里没有 `item` ——直接抛 `ReferenceError` 把徽章渲染打断。是"改完必须真跑一遍"抓到的。
- 修好后 `--dump-dom` 突然失效 (输出 0 KB) , 我**无法证明修复有效**, 于是按规矩 **先撤回**, 把它记成"待浏览器复核"; 后来发现 `--print-to-pdf` 与 `--screenshot` 仍可用, 换用**截图验证**才重新启用。说不清能不能跑通的, 就不说它能跑通。

前提有误的三条 (已核实并驳回)

「**首屏示例** `ps aux | grep nginx | ... | xargs -r kill -HUP` 没标 [模拟]」首屏示例根本不是这条, 是「找出当前目录下最大的 10 个文件」 → `du -ah . | sort -rh | head -n 10` , 而 `du` 是**完整真实实现** (实测 exit 0、11 行真实输出)。它建议的"换成 `find/ls/du` 做首屏演示" **早就是这样了**。另外实测把 `ps` 那条单独丢进沙箱, `simulated` 标志确实被置起、警示条确实会出——它担心的"没标注"不成立。

「**核心亮点自验证闭环首屏无感知**」首屏那条绿色徽章「沙箱自验证通过 · 第 1 次就真跑通 (未触发修正)」就在折叠线内 (实测 $y=821 < 842$) 。这正是前几轮专门改的东西, 它没看到。

「**输入框上方写着『不用配 Key、不用联网』**」那句话在「**关于这个作品**」**抽屉的 hero** 里, 不在输入框上方。输入框上方只有「你想让电脑做什么」和快捷键提示。

成立的三条 (已修)

① **快捷键提示不够明确** X 「`Ctrl + ↵`」只说了怎么提交, 没说回车干什么。改成「**Ctrl + ↵ 生成 · ↵ 换行**」——两件事一次说清。

② **hero 那句表述可以更严谨** X 原文「不用配 Key、不用联网也能完整跑通」在语义上没错（离线确实能完整跑通），但容易被读成"AI 也不用联网"。收紧为「**内置示例免 Key、免联网；AI 生成需联网（密钥在服务端，前端不留）**」。

③ **上标序号——但它指出的问题比它说的严重** X X 角标不是 CSS 伪元素，是**命令文本里的真实节点**：`<span class="tok">片段<span class="tok__mark">2</span></span>`。所以问题不是"可能被误读"（它其实悬浮在片段上方 1.4em，视觉上并不混淆），而是**手动框选复制命令时会把序号一起复制走**——复制出来是 `ps -o pid,rss,cmd1 -C nginx2 --sort=-rss3`，**一条坏命令**。复制按钮走的是干净的 `item.cmd` 不受影响，但框选会中招。修法是给它 `user-select: none`：一行 CSS，问题根除。

（顺带自我更正：我早先做探针时读到过 `du -ah .1 | sort -rh2 | head -n 103`，当时判断"是探针的选择器错了"。**那个判断是错的**——DOM 里真的长那样，只是角标被 `display:none` 藏着看不见。是这份评审让我回头复查才发现的。）

另外补的一处（评审没提，但同类）

模拟命令的警示条原先只在执行区（输出**下面**）。而"这条命令含模拟成分"这件事，**应当在看到命令时就成立**。所以命令区也标一处：`SYNOPSIS · 命令` 旁边加一枚琥珀色「含模拟数据」小标，实测与执行区那条同时出现；首屏那条真实命令则两处都不出。

实现上踩了一个坑：小标最初写在 `runHtml()` 里查 `#secSyn`——而 `runHtml` 只是**拼字符串**，卡片那时还没进 DOM，轻则查不到、重则**挂到上一张卡上**。挪到 `paintRun()` 里（结果已出、卡片已在 DOM），并用卡片元素限定范围后才正确。

实测证据：

```
`` 问：把项目里所有 png 图片转成 webp 答：find . -name '*.png' -exec cconvert {} {} \.webp \; ← cconvert 是编的，沙箱里叫 cwebp 问：把 uploads 里的 png 改名成 1.png 2.png 答：ls uploads | ... | while read f; do ... done ← 沙箱没有 while 循环 ``
```

这跟"占位符"是**同一个根因**：提示词描述的世界与沙箱的真实世界脱节。模型不是在犯错，它是在一个它看不见的沙箱外面猜。

解法：沙箱自己有一条 `help` 命令，输出正好是一份准确的能力清单（命令 + 每个命令支持的参数写法）。把它注入 system 消息即可——而且这份清单来自沙箱自身，**永远不会和实现漂移**（比手写一份常量可靠）。

实测效果（同一批 5 条需求，注入前后）：

| 指标 | 注入前 | 注入后 | |---|---|---| | 首次就跑通 | 3/6 | **4/5** | | 用了沙箱没有的命令/语法 | 1 个 | **0 个** | | "把图片转 webp" | `cconvert`（编的）X | `cwebp -q 90`（真的有）✓ |

约 33% → 约 80% 的首次通过率。这条改动的价值比"让闭环更容易被撞见"大得多——因为闭环之所以撞不见，本来就是因为第一次大多能过；而第一次过不了的那些，现在也大多能过了。**真正该修的是"让它一次就对"，不是"让它更容易出错"**。

（过程中的一次事故，如实记下：我用字符串索引插入导出语句时，`IndexOf` 返回了 -1，我又拿它加了长度，结果插到了**文件开头**，把第 2 行的注释切成了两半。是紧接着的语法检查与 `Select-String` 核对发现的，已修回。**教训同前：用索引插入前必须先断言锚点存在。**）

```
`` 第 1 轮生成 → 送沙箱 → 失败则把真实报错 + 沙箱能力清单喂回 → 第 2 轮再跑 ``
```

结果挖出三个真问题，而且第一个比"看不见"严重得多：

① **提示词在教模型写占位符** X X 模型输出了 `rm -rf <cache_directory>`，追下去发现 `SYSTEM_PROMPT` 第 3 条原文是「需要用户自己填的地方用 `<占位符>`」——**不是它不听话，是它在执行我的指令**。而"拿到一条能直接粘贴执行的命

令"正是这个作品的立身之本，带尖括号的模板等于交了半成品。两处提示词（长版与极客短版）都已改成"不许占位符，缺路径就用当前目录并说明假设"。验收指标：实测 6 条需求的生成结果里，占位符数量 0。

② 沙箱缺 `grep --include` X 它被当成普通位置参数，于是报出"无效的正则：*.js"——一个跟真实原因毫不相干的错，模型看了也改不对。而这是真实 `grep` 最常用的参数之一。现在三种写法都认（`--include=*.js` / `--include '*.js'` / `--include='*.js'`），`--exclude` 一起补上，并断言真的按扩展名过滤（只认参数不生效等于没实现）。沙箱测试 97 → 103 项。

③ `grep` 没匹配时返回 1，被当成了失败 X `grep -R TODO src` 在没有匹配时退出码是 1——那是 `grep` 的正常语义，可自验证把它当失败，白触发一轮修正、闭环徽章还显示"未通过"。`diff` 同理。现在这两种命令"退出码 1 + `stderr` 干净"判定为"跑通了，只是没有结果"。

诚实的结论：一次通过从 2/6 升到 3/6（占位符修复带来的），但"修正后通过"仍是 0/3——剩下的失败都指向"沙箱缺少所需语法"（`shell` 的 `while` 循环、`awk` 的求和）。这两项要真正支持是另一个量级的工程，所以我把结论写在这里：闭环的"修正"那一半，目前只在"路径写错、参数写错"这类简单错误上可靠；遇到"沙箱根本不支持这个语法"会修不好。把它当作一个已知局限，而不是继续包装成"杀手铜"。

新方向是「工程图纸」，理由是**这个站本来就有图纸语言**：命令解剖用引线标注、风险用印章、章节用 `SYNOPSIS` · 命令 这种编号标签、分隔线是发丝线。这些早就在了，只是被那层通用开发工具的皮盖住了。所以改的是"纸"，不是"字"：

| 元素 | 前 | 后 | ---|---|---| | 底色 | 深紫 `#1e1e2e` | 深海军蓝 `#0b1220` | | 细线 | 青色半透明 | 近白色发丝线（图纸感的关键在"白线"） | | 圆角 | 8px | 2px | | 阴影 | 柔和投影 | 锐利描边 + 深投影 | | 背景 | 单一网格 | 细格 + 每五格加粗（工程图纸的标配） | | 卡片 | 一 | 四角裁切标记（纯背景层，不占一像素布局） | | 页脚 | 一行版权 | 图纸标题栏：图名 / 图号 / 赛道 / 设计 / 日期 / 比例 / 许可 |

语义色（安全绿 / 谨慎琥珀 / 危险红）一个都没动：它们要能被一眼读懂，不该为了"风格统一"而牺牲含义。强调色也保留青色——换成橙色会和"谨慎"撞色，那是拿可读性换风格。

这次重塑之所以**风险低**，是因为前期那条纪律：所有色值集中在 `:root`，改完全表只动了十来行令牌（实测 `:root` 之外只有 4 处真硬编码色值）。

之前做"首屏即产品"时，我把落地页整块收进抽屉，理由成立（首屏该是产品、不该是营销）。但我把**"这是什么"和"营销文案"一起删掉了**：首屏顶栏当时只说得「赛道三·智能交互与应用」，那是**参赛分类**，不是**产品是什么**。评委得自己从输入框和一张命令卡去推断——这在两分钟的打分节奏里是实打实的损失。

但整块 hero 也不能搬回去：外部评审的原话是"首屏堆了五六个层级、一进来不知道先看哪"，把大标题 + 三张亮点卡 + 按钮 + 演示窗整套放回首屏，**正好把那句话说的事情做一遍**。而且那三张卡讲的是"能做什么"，首屏那张**真命令卡**已经在替它们说话了。

所以改法是**紧凑的身份标识**，只留两行、不搬 hero：

```
`` NL2SH Linux 新手零门槛入门平台 中文说需求 → 真模型生成 → 沙箱真跑一遍 → 一键回滚 赛道三 · 智能交互与应用 · GDUST Linux 协会 AI 网页设计大赛参赛作品 ``
```

作品名用 19px 亮色（`--ink`），流程那句推右侧用小字，赛道归属单独占第二行——**三层的视觉重量依次递减，不与产品本身抢注意力**。

实测代价与收益：标识栏 43 → 69px（+26px），输入框 191 → 217，结果卡 427 → 453，**闭环徽章 795 → 821（仍在 842 折叠线内 ✓）**，目录树 952（折叠线下 110px）。

这条改动的价值不在像素，在于把"这是什么"从"需要推断"变成"一眼看到"——而代价只有 26px，没有把页面变回落地页。

先逐层量了一遍"谁占着首屏"（不猜，量出来的）：

| 位置 | 高度 | 判断 | |---|---|---| | 卡头（含风险印章 + 风险说明） | 149px | 印章是装饰性的，占了近 90px | | 沙箱状态条 + 欢迎说明条 | 52 + 43 = 95px | **两处都在解释"沙箱是真的、改动在内存里"** | | EXECUTION 说明段 | 24px / 78 字 | 与上面两条又是同一件事，**三处重复** |

改动三处：① **风险印章收成"一枚小戳"**：84×58 → 60×37，字号 16 → 13。语义一点没少（颜色 + 三个字 + 缩写），但它不再和"命令本身"抢版面。卡头 149 → **122px**。② **欢迎条只说它独有的事**：原来在解释沙箱真实性（状态条已经在说），现在只讲"这是给你的起点，改成自己的需求回车就走真实模型"。③ **EXECUTION 说明段收到一句**：78 字 → 24 字，只保留独有信息（可回滚）。

实测：命令框 y=683 → **656**，闭环徽章 y=823 → **795（第一屏内）**，目录树 y=953 → **926**。

收益比预期小，所以到此为止：目录树还差 84px 进折叠线，但要再往下就得合并"沙箱状态条 + 欢迎条"这两个已经稳定的结构——为了 84px 去动它，风险大于收益。在 1080p 屏上目录树本来就在第一屏内；在 900px 笔记本上它差一次小滚动，而第一屏已经有：赛事标识、输入框、沙箱状态、真命令卡、命令、闭环徽章。**该收手的地方就收手。**

七、视觉方向：工程图纸，两种信息密度

主题是「Linux 原生命令行」，但视觉语言选的是「工程图纸」而不是"又一个深色开发工具"。

这个选择不是硬套风格，而是**把站内本来就有的语言做彻底**：命令解剖用引线标注、风险判定用印章、章节用 `SYNOPSIS · 命令` 这种编号标签、分隔线是发丝线——这些从第一版就在，只是原先被"深紫底 + 青色高亮 + 圆角卡片"那层通用皮盖住了。

为什么必须换：60% 的权重看的是"视觉风格与品牌感"，而旧版用的 Catppuccin Mocha（深紫底 + 青色高亮）是满屏开发者工具的标准配置——干净、统一，但**记不住**。

令牌	色值	用在哪
底色	<code>#0b1220</code>	页面底（深海军蓝）
面板	<code>#101a2b</code> / <code>#16223a</code>	卡片 / 内嵌层
细线	<code>rgba(232, 238, 248, .16)</code>	近白发丝线——图纸感的关键在"白线"，不在颜色
强调	<code>#7ad7ea</code>	强调、链接、当前项
安全绿	<code>#7ee787</code>	安全等级、成功态、可执行权限
谨慎 / 危险	<code>#f9e2af</code> / <code>#ff7b8a</code>	谨慎 / 危险

语义色一个都没动。安全绿、谨慎琥珀、危险红要能被一眼读懂，不该为了"风格统一"而牺牲含义；强调色也保留青色系——换成橙色会与"谨慎"撞色，那是拿可读性换风格。

几何与纹理上做了六件事：

- 底色从深紫换成深海军蓝**：紫调深灰是"编辑器主题"的观感，海军蓝才像图纸。
- 细线从青色半透明换成近白发丝线**：这是整套视觉里最容易辨认的一处改动——图纸之所以像图纸，是因为线是白的。
- 圆角 8px → 2px**，柔和阴影 → 锐利描边 + 深投影：**图纸上的框是画出来的，不是浮起来的**。
- 背景改成"细格 + 每五格加粗"**：工程图纸最容易被认出来的特征，成本只是多一层渐变。
- 卡片四角加载切标记**：纯背景层实现，**不占一像素布局**——所以首屏那些量好的位置一个都没动。

6. **页脚加图纸标题栏**：图名 / 图号 / 赛道 / 设计 / 日期 / 比例 / 许可。它是这套语言里辨识度最高的一格，而页脚天生不占首屏。

配色令牌全部集中在 `:root`，**样式表里除 `:root` 外只有 4 处真硬编码色值**。所以这次重塑只动了十来行令牌，而不是全表手术——这是前期那条纪律的直接回报。

另外还有三件从第一版保留至今的做法：

- 1. **正文用偏冷的浅灰而不是纯白**：纯白配深底在长文阅读下发涩。
- 2. **图标全部自绘 SVG，不用 emoji**。三张亮点卡的图标（提示符→输出、管道段、沙箱立方体）是 24×24 描边图形，跟正文同一套线宽与颜色，各自带一块染色底板。emoji 的问题不只是"廉价"——它的粗细、色彩、基线都不受控，在深色底上会跳出来抢戏，而且换个系统就换一张脸。
- 3. **粒子场的密度调到"看得见但不抢戏"**：字符场对应命令行文本、管道流带对应数据流方向、执行成功/失败用不同颜色爆散、回滚时粒子回流到按钮上。**每一个动效都指向一个真实状态**，不是装饰。

两个模式：配色同源，但界面不同。新手引导是完整落地页 + 教学；极客终端把落地页与教学整块收起，底色压暗一档、细线转绿、顶部挂模式横幅——切换后首屏内容完全不同，一眼能分辨（详见第六节第 12 条）。

八、工程验证：1468 项自动化检查 + 部署后真机自测

这是工程上最花时间的部分，也是"能不能改得动"的保证。

测试文件	项数	覆盖内容
<code>scripts/sandbox-test.mjs</code>	103	命令语义全部真跑一遍并对输出与副作用做断言；权限拒绝；快照回滚；glob 引号感知； <code>\$(...)</code> ； <code>for</code> 循环； 确定性 （同一命令三次结果必须完全一致，且 fixture 源码里不许出现 <code>Math.random</code> / <code>Date.now</code> ）
<code>scripts/data-test.mjs</code>	1255	规则库每一条 ：结构完整性、解释片段必须是命令的真实子串且互不重叠、命令名在沙箱里确实实现、 整条命令真跑一遍 （退出码、有无输出或文件变更、stderr 是否干净）、问句与命令唯一性
<code>scripts/ai-test.mjs</code>	59	JSON 萃取（围栏 / 废话 / 字符串内花括号 / 半截）； 非法转义修复 （真机抓到的坏 payload 原文 + <code>\\" \n \\ \uXXXX</code> 不被误伤）；新结构归一化；重叠过滤；危险命令改判；修正请求是否真的带上了沙箱报错；用 <code>ReadableStream</code> 模拟真实 SSE 流；HTTP 错误分类
<code>scripts/worker-test.mjs</code>	51	边缘代理：密钥注入、模型白名单、 <code>response_format</code> 强制开启、 <code>max_tokens</code> 上限、SSE 头、413/400/405、限流、上游透传
<code>scripts/ai-live-test.mjs</code>	16	部署后真机自测（需要网络） ：加载真实 <code>ai.js</code> → 走它自己的端点回退 → 打已部署的 Worker → 真实调 <code>glm-4-flash</code> → 断言调用凭证（模型名 / 首字节 / 总耗时 / 分片数 / 端点）→ 把模型给的命令 送进沙箱真跑

前四套**不需要联网、不需要 Key**；最后一套是部署后跑一次的验收，跑法：

```
node scripts/sandbox-test.mjs
node scripts/data-test.mjs
node scripts/ai-test.mjs
node scripts/worker-test.mjs

# 部署后验收（需要网络；国内直连 workers.dev 不通时走代理）
node --use-env-proxy scripts/ai-live-test.mjs
```

node scripts/worker-test.mjs

测试真的抓到了东西（不是摆设）

| 缺陷 | 现象 | 根因 |

| ---|---|---|

| `du` 看不到整个 `node\_modules` | 目录合计少了 1.9 MB | 场景漏建了 `node\_modules` 目录节点，而 `children()` 按前缀找直接子项，整棵子树成了孤儿 → **顺手加了一条"孤儿节点"测试** |

| `ps --sort=-rss` 输出垃圾字符串 | 表头变成 `RT=-RSS` | `parseArgs` 的长选项分支写成了 `a[0] === '--`（单字符永远不等于两字符），是**死代码** |

| `for` 循环里 `cwebp` 报 "80: No such file" | 压缩质量被当成输入文件名 | `-q` 被放进"布尔开关"位而不是"带值选项"位 |

| `rm -rf /\*` 没被危险模式拦住 | 只拦住了 `rm -rf /` | 正则要求 `/` 后是空白，被 `\*` 绕过 → 改成**拆开判断**（有没有递归开关 + 目标是不是根） |

| 整个页面白屏 | 首屏、状态、卡片全没了 | 少一个右括号，**一个语法错误让 `app.js` 全盘不执行** → 现在测试第一步就是语法检查 |

| 加载时就多出一条空白提示栏 | 底部有个空的「结束演示」按钮 | `.demoBar { display: flex }` 盖掉了 `hidden` 属性 → 改成全局 `[hidden] { display: none !important }` |

| 演示时页面不滚动 | 解说条在讲，画面却不动 | `scrollIntoView()` 会去挑"最近的可滚动祖先"，行为不稳定 → 改成算绝对偏移 + `window.scrollTo` |

| **`du` 总量每次都不同** | 同一路径在"原样"与"排序后"两次运行里是 2.1M / 1.7M | 场景文件大小用了 `Math.random()` → 整个沙箱失去确定性，也让"可复现"变成假话 |

| **模型带正则的回答整个解析失败** | 界面上什么都出不来 | 模型把 shell 的 `\\(` `\\)` 直接写进 JSON 字符串，而 JSON 不认这种转义。**1468 项离线断言全用了合法 JSON，只有真机打模型才照出来** → 加了一层"解析失败时修非法转义再重试"，并补 8 项回归测试 |

三个方法论教训（值得写下来）

1. **无头浏览器的虚拟时间不产帧**。`chrome --headless --virtual-time-budget``
只推进定时器，不推进 CSS transition、`requestAnimationFrame``，也不真正应用滚动。
结果是页面看着"半成品"、演示看着"不滚动"，全都是假象——真机上完全正常。
涉及动画、滚动、rAF 的验证必须用真实时间。
2. **验证部署必须用真实 GET，不能用 HEAD**。用 HEAD 检查过腾讯云 COS 的响应头，没看到 `Content-Disposition`` 就以为配置对了；真实 GET 才看到 `x-cos-force-download: true``（对方默认域名强制下载）。
3. **尺寸是最后一道便宜的保险**。给 `<script>` 加上 `defer`` 之后，构建脚本的内联正则只认 `<script src= "... ">`，**7 个脚本一个都没被内联**，单文件从 281 KB 缩到 50.9 KB——而当时的守卫用的是同一个正则，所以它没报错，构建"成功"了。
如果没核对产物大小，交上去的就是一个只有骨架、功能全无的页面。
现在守卫改成"找还有没有本地资源引用"，并加了一道尺寸断言。
4. **"命令没报错"不等于"产物出来了"**。生成说明书 PDF 与页面截图时，有几次命令静默失败：一次是 PowerShell 把数组里的 `'--window-size=880,' + $n`` 拆成了两个参数，多出来的数字被 Chrome 当成第二个目标，报 `Multiple targets are not supported in headless mode`` 后什么都不产出；一次是进程还没写完文件就被清理掉。
现在所有产物生成后都**先确认文件真的存在且大小稳定**，再继续下一步。

九、部署与上线实测

比赛的提交物是**一个作品链接**（报名表里那一列），所以站点必须挂到一个**点开就是网页**的公网地址上。

一个踩过的坑

最开始选的是**腾讯云 COS 静态网站**，按官方文档配置完、本地测也通了，

但**链接点开是下载文件**。原因是产品策略：

2024 年 1 月起 COS 默认域名访问任何文件都不支持预览、直接下载，响应头会带 `x-cos-force-download: true`；官方唯一解法是绑定自定义域名，而自定义源站域名要求「域名已备案」，备案要 1-3 周。**直接否决**。

最终方案：**GitHub Pages**。线上地址 <<https://muxinbo.github.io/nl2sh/>>

上线后的实测记录（真实 GET，不是 HEAD）

检查项	结果
--- ---	
HTTP 状态	200
`Content-Type`	`text/html; charset=utf-8`
`Content-Disposition`	**不存在** → 浏览器正常渲染网页
`http://` 访问	301 强制跳 HTTPS
响应耗时	总 0.387s / 首字节 0.212s
窄屏渲染	375px 下无横向滚动，亮点卡正确降为单列

其它结论

1. **必须 HTTPS**：`localStorage`、剪贴板 API 在非安全上下文里会被限制，页面上的「复制」按钮会退化。
2. 站点资源路径**全部是相对路径**，放在子目录（如 `https://user.github.io/nl2sh/`）也能正常工作。
3. **单文件产物是主推形态**：不会出现“漏传 `assets/` 导致白屏”这类最容易发生的部署事故。

三种提交形态都准备好了：

形态	大小	适用场景
--- ---	--- ---	
单文件 `dist/index.html`	332.8 KB	拖放部署、单文件上传，推荐
多文件目录（`index.html` + `assets/`）	-	传 Git 仓库、给评委看源码
打包 `dist/nl2sh-多文件版.zip`	2.3 MB	虚拟主机 / 宝塔面板上传解压

十、评分维度逐条自查与已知局限

10.1 逐条自查

维度	权重	做了什么	证据
--- ---	--- ---		
内容与主题	20%	场景收窄到“不会敲命令的新手”；解释讲“为什么”；危险命令给更安全的替代；模型说错话时明确标注来源；模拟输出标注 `[模拟]`；规则库 50 条全部人工核对参数语义与风险等级	第八节 1255 项规则库检查；本文档第二、三节
技术与运维	10%	零依赖零构建；流式 SSE；同源边缘代理 + 密钥只在服务端 + 离线降级；自研 shell 与内存文件系统（权限 / 快照 / diff / **确定性**）；危险命令双层拦截	1468 项自动化检查 ；`worker-test.mjs` 51 项
创意设计与交互体验	60%	第六节 34 条；首屏自动演示窗； AI 调用凭证 （模型 / 首字节 / 流式分片 / 端点）；生成中的实时流式证据条；沙箱自验证自纠错闭环；管道节点图 + 真实分段样例；实时权限染色目录树；删除红闪 + 一键回滚；高危预警前置；复制闸门；双模式（两个界面）；8 个示例；终端逐行显形 + 收藏；深色终端配色	第六节；`preview/` 目录效果图；真机自测 `ai-live-test.mjs` 16 项
作品价值	10%	解决“不知道敲什么”和“不敢敲”；免费方案让作品能长期在线；沙箱是**可以放心试错的练习场，也可直接用于学校 Linux 课程演示	第一节定位；第四节沙箱语义清单

10.2 已知局限（主动写出来，不含糊）

1. **沙箱不是真 Linux**：`systemctl` / `journalctl` / `ss` / `ps` / `kill` / `cwebp` 是固定场景生成，界面上都标注了 `[模拟]`。真内核方案（WASM）因体积与 `SharedArrayBuffer` 限制被否决，理由见第四节。
2. **规则库覆盖有限**：50 条覆盖常见运维与文件操作，长尾需求必须依赖模型；

无网时只能走规则库。

3. ****依赖模型时会出错****：三层防护能挡住"危险命令"和"结构错位"，但挡不住"解释得不够准"。AI 生成的卡片明确写着「参数语义请自己再核一遍」。
4. ****缺一次真实用户可用性验证****：目前只有开发者自测与自动化检查，没有找目标用户（大一新生）做过正式的可用性测试。
5. ****推演看不到读操作****：影子沙箱的 `diff` 只能给出被改动的路径，给不出"读了哪些文件"（见第三节边界说明）。
6. ****`glm-4.7-flash` 不可用****：规格里提到的这个模型 ID 实际不存在，已按要求降级到其指定的 `fallback `glm-4-flash``，实测数据见 5.3。

10.3 后续可拓展方向

1. 多发行版适配（`apt` / `dnf` / `pacman`）
2. 错题本：把生成过但没看懂的命令收起来复习
3. 沙箱场景可选（Web 服务器 / 数据分析 / 刚装好的新机器）
4. 新手闯关扩展到 10 关（目前 5 关，判定机制已经是"看沙箱状态"，加关只需加判定函数）

附录 A：目录结构

nl2sh/ |— index.html 页面结构 |— assets/ | |— styles.css 配色令牌、全部样式、响应式 | |— data.js 内置规则库（50 条已核对并真跑验证） | |— sandbox-fixtures.js 沙箱场景（一个虚构但自治的学生服务器） | |— sandbox-cmds.js 48 个命令的真实语义实现 | |— sandbox.js 内存文件系统 + 迷你 shell + 快照/diff 引擎 | |— ai.js 模型接入层（只请求同源 /api/chat，前端没有密钥） | |— fx.js 粒子层（字符场 / 管道流 / 执行爆散） | |— app.js 界面逻辑（管道节点 / 目录树 / 闭环 / 演示） |— worker/ | |— worker.js Cloudflare Worker 边缘代理 | |— wrangler.toml 部署与限流配置 |— scripts/ | |— sandbox-test.mjs 沙箱自测 103 项 | |— data-test.mjs 规则库自测 1255 项 | |— ai-test.mjs 模型层自测 59 项 | |— worker-test.mjs 代理层自测 51 项 | |— build-single.mjs 把全站内联成单文件 |— deploy/ 部署文档（Worker / GitHub Pages / Nginx 与踩坑记录） |— dist/index.html 单文件产物，332.8 KB，双击即用 / 单文件上传 |— preview/ 效果图 |— README.md 仓库首页 |— 作品说明.md 本文档

附录 B：AI 在本项目里的角色

- ****架构决策来自实测而不是猜测****：模型可用性、流式首字节、JSON mode 的收益、跨域可达性，全部先发请求验证再定方案。"`glm-4.7-flash` 不存在"（500 / 75.9s）、"推理模型太慢且会包围栏"（28s / 303 个 SSE 事件）、"WASM coreutils 太重"（12.4 MB + `file://` 取不到）都是实测推翻的方案。
- ****"两个模式 = 两份输出预算"**** 是被"免费额度下首字节 0.43s / 完整回答 6.35s"这个测出来的物理限制逼出来的。
- ****沙箱的取舍是被约束逼出来的****：要秒开、离线、U 盘可带走，就只能自己实现语义，并且明确划出"哪些是模拟的"。
- ****八个真实缺陷由自动化测试与实测发现并修复****（见第八节表格），其中"`du`` 总量每次不同"是这一轮做管道样例时被自己的新功能照出来的。
- ****纠正过两次自己的验证方法****：一次是无头浏览器虚拟时间不产帧造成的"页面坏了"假象；一次是把探针文件放在项目目录之外，导致相对资源 404，得出了错误的"样式没加载"结论。两条都写进文档，避免下次再被同一个假象误导。
- 视觉方向由人工在候选里拍板，否掉了最像模板的两个方案。
- 内置规则库 50 条由 AI 起草，逐条人工复核参数语义与风险等级，并由 `data-test.mjs` 全部真跑一遍。